

Vorlesung 8: Kondition & Pivotisierung

§3.5 Störungssatz | §3.6 Permutationsmatrizen & LR mit Spaltenpivotsuche

1 §3.5 Störungstheorie: Kondition und Fehlerfortpflanzung

Wiederholung — Konditionszahl und Problemstellung

Für eine reguläre Matrix $A \in \mathbb{R}^{n \times n}$ und eine submultiplikative Matrixnorm $\|\cdot\|$:

$$\text{cond}(A) := \|A\| \|A^{-1}\|.$$

Gegeben: $Ax = b$ (exakt) und $(A + \Delta A)(x + \Delta x) = b + \Delta b$ (gestört).

Ziel: Schranke für den relativen Lösungsfehler $\|\Delta x\|/\|x\|$ in Abhängigkeit von relativer Matrixstörung $\|\Delta A\|/\|A\|$ und relativer Datenstörung $\|\Delta b\|/\|b\|$.

Neumann-Reihe (Hilfsmittel): Falls $\|B\| < 1$ (submultiplikative Norm), dann ist $(I - B)$ invertierbar und

$$(I - B)^{-1} = \sum_{k=0}^{\infty} B^k, \quad \|(I - B)^{-1}\| \leq \sum_{k=0}^{\infty} \|B\|^k = \frac{1}{1 - \|B\|}.$$

Satz 3.32 — Störungssatz

Sei $\|\cdot\|$ eine submultiplikative Matrixnorm, $A \in \mathbb{R}^{n \times n}$ regulär, und

$$\gamma := \|\Delta A\| \|A^{-1}\| < 1.$$

Dann ist $\hat{A} = A + \Delta A$ ebenfalls regulär und für den relativen Lösungsfehler gilt:

$$\frac{\|\Delta x\|}{\|x\|} \leq \frac{\text{cond}(A)}{1 - \gamma} \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta b\|}{\|b\|} \right).$$

Beweisschritte:

1. Setze $B := \Delta A \cdot A^{-1}$. Dann: $A + \Delta A = (I + B)A$, $\|B\| \leq \|\Delta A\| \|A^{-1}\| = \gamma < 1$.
2. *Neumann-Reihe:* $(I + B)$ invertierbar, $\|(A + \Delta A)^{-1}\| \leq \|A^{-1}\| \cdot \|(I + B)^{-1}\| \leq \|A^{-1}\| \cdot \frac{1}{1 - \gamma}$.
3. *Fehlergleichung:* Aus I.) $Ax = b$ und II.) $(A + \Delta A)(x + \Delta x) = b + \Delta b$ folgt $(A + \Delta A) \Delta x = \Delta b - \Delta A x$, also $\|\Delta x\| \leq \|(A + \Delta A)^{-1}\| (\|\Delta b\| + \|\Delta A\| \|x\|)$.
4. Abschätzen und $\|b\| \leq \|A\| \|x\|$ einsetzen liefert die Behauptung. $\square$

Kommentare zum Störungssatz

Für $\gamma \ll 1$: $\frac{1}{1 - \gamma} = 1 + O(\gamma) \approx 1$, also vereinfacht sich die Schranke zu $\frac{\|\Delta x\|}{\|x\|} \lesssim \text{cond}(A) \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta b\|}{\|b\|} \right)$. Die Konditionszahl gibt direkt den Fehlerverstärkungsfaktor an.

Schärfe: Die Schranke ist im Allgemeinen scharf. Für SPD-Matrix $A \in \mathbb{R}^{n \times n}$ mit Eigenwerten $0 < \lambda_1 \leq \dots \leq \lambda_n$ und ONB $v_1, \dots, v_n$: wähle $\Delta A = 0$, $b = v_n$, $\Delta b = \varepsilon v_1$. Dann $x = \lambda_n^{-1} v_n$ und

$\Delta x = \varepsilon \lambda_1^{-1} v_1$, also

$$\frac{\|\Delta x\|}{\|x\|} = \frac{\lambda_n}{\lambda_1} \cdot \frac{\|\Delta b\|}{\|b\|} = \text{cond}_2(A) \cdot \frac{\|\Delta b\|}{\|b\|}.$$

Zwei unabhängige Fehlerquellen: (1) Schlechte Kondition von A — lässt sich nicht ändern. (2) Numerische Instabilität des Lösungsalgorithmus — durch Pivotisierung behebbar.

2 §3.6 Pivotstrategien

Beispiel 3.33 — Instabilität der LR-Zerlegung ohne Pivotsuche

In 4-stelliger Dezimalarithmetik $\mathbb{F}(10, 4)$ mit $A = \begin{pmatrix} 3.1 \cdot 10^{-4} & 1 \\ 1 & 1 \end{pmatrix}$, $b = \begin{pmatrix} -3 \\ -7 \end{pmatrix}$.

Ohne Zeilentausch: $l_{21} = 1/(3.1 \cdot 10^{-4}) \approx 3.226 \cdot 10^3$, $R \approx \begin{pmatrix} 3.1 \cdot 10^{-4} & 1 \\ 0 & -3.225 \cdot 10^3 \end{pmatrix}$.

Lösung: $\hat{x} = (-3.226, -2.999)^\top$ vs. exakt: $x = (-4.001246, -2.99875)^\top$.

Erster Eintrag hat Fehler $\approx 19\%$ — **massiv falsch**.

Mit Zeilentausch (Zeile 1 $\leftrightarrow$ Zeile 2): $l_{21} = 3.1 \cdot 10^{-4}$ (klein!), $R_{per} \approx \begin{pmatrix} 1 & 1 \\ 0 & 0.9997 \end{pmatrix}$.

Lösung: $\hat{x} = (-4.001, -2.999)^\top$ — korrekt bis auf Rundungsgenauigkeit.

Ursache: Das kleine Pivot $3.1 \cdot 10^{-4}$ erzeugt großen Multiplikator $l_{21} \approx 3226$, was zu massiver Auslöschung führt. Die Matrix ist gut konditioniert ($\text{cond}(A) \approx 3$) — das Problem liegt *ausschließlich* am Algorithmus.

Definition 3.35 — Permutationsmatrix

Sei $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ eine Permutation (bijektiv) und $e_1, \dots, e_n$ die Standardeinheitsvektoren: $(e_i)_j = \delta_{ij}$.

Die **Permutationsmatrix** zu π ist:

$$P_\pi := (e_{\pi(1)} \mid e_{\pi(2)} \mid \dots \mid e_{\pi(n)}) \in \mathbb{R}^{n \times n}.$$

Spalte j von P_π ist $e_{\pi(j)}$; Eintrag $(P_\pi)_{ij} = \delta_{i, \pi(j)}$.

Lemma 3.36 — Eigenschaften von Permutationsmatrizen

Sei π eine Permutation, P_π zugehörige Permutationsmatrix. Dann:

1. $P_\pi e_i = e_{\pi(i)}$ für $i \in \{1, \dots, n\}$.
2. $P_\pi^{-1} = P_\pi^\top$ (Permutationsmatrizen sind **orthogonal**).
3. Die i -te Zeile von $P_\pi A$ ist die $\pi(i)$ -te Zeile von A (Linksmultiplikation = **Zeilenpermutation**).
4. Für weitere Permutation $\hat{\pi}$: $P_{\hat{\pi} \circ \pi} = P_{\hat{\pi}} P_\pi$.

Beweis: Übungsaufgabe.

Beispiel (Zeilentausch): $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = \begin{pmatrix} a_{21} & a_{22} \\ a_{11} & a_{12} \end{pmatrix}$.

Algorithmus 3.37 — LR-Zerlegung mit Spaltenpivotsuche

Input: Reguläre Matrix $A \in \mathbb{R}^{n \times n}$. **Output:** Zerlegung $PA = LR$ mit Permutationsmatrix P .

Pseudocode:

1. $A^{(0)} := A$.
2. **for** $k = 1, \dots, n - 1$:
 - *Pivotsuche:* Finde $i \in \{k, \dots, n\}$ mit $|a_{ik}^{(k-1)}| = \max_{i' \in \{k, \dots, n\}} |a_{i'k}^{(k-1)}|$.
 - *Zeilentausch:* Definiere Permutation π_k (tauscht Zeile $i \leftrightarrow k$), ersetze $A^{(k-1)} \leftarrow P_{\pi_k} A^{(k-1)}$.
 - *Eliminationsschritt:* Berechne Multiplikatoren $l_{ik} = a_{ik}^{(k-1)} / a_{kk}^{(k-1)}$ für $i = k + 1, \dots, n$.
 - Setze $A^{(k)} := L_k A^{(k-1)}$.
3. $\pi := \pi_{n-1} \circ \dots \circ \pi_1$, $P := P_\pi$, $R := A^{(n-1)}$.

In der Praxis (Python): Statt Zeilen physisch zu vertauschen wird ein Permutationsvektor p geführt; Zugriff auf permutierte Zeile (i, j) erfolgt via `A[p[i], j]`.

$$\text{Bsp.: } \pi(1) = 3, \pi(2) = 1, \pi(3) = 2 \Rightarrow p = \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix} \quad (0\text{-indiziert})$$

Beispiel — LR mit Spaltenpivotsuche (3×3)

$$A^{(0)} = \begin{pmatrix} 1 & 2 & 2 \\ 2 & -7 & 2 \\ 1 & 24 & 0 \end{pmatrix}.$$

Schritt 1 ($k = 1$): Pivot in Spalte 1: $|a_{21}| = 2 > |a_{11}| = 1$, also Tausch Zeile $1 \leftrightarrow 2$.

$$\tilde{A}^{(0)} = \begin{pmatrix} 2 & -7 & 2 \\ 1 & 2 & 2 \\ 1 & 24 & 0 \end{pmatrix} \Rightarrow A^{(1)} = \begin{pmatrix} 2 & -7 & 2 \\ 0 & 5.5 & 1 \\ 0 & 27.5 & -1 \end{pmatrix}.$$

Schritt 2 ($k = 2$): Pivot in Spalte 2: $|a_{32}| = 27.5 > |a_{22}| = 5.5$, Tausch Zeile $2 \leftrightarrow 3$.

$$\tilde{A}^{(1)} = \begin{pmatrix} 2 & -7 & 2 \\ 0 & 27.5 & -1 \\ 0 & 5.5 & 1 \end{pmatrix} \Rightarrow A^{(2)} = R = \begin{pmatrix} 2 & -7 & 2 \\ 0 & 27.5 & -1 \\ 0 & 0 & 1.2 \end{pmatrix}.$$

$$\text{Permutationsmatrix: } P = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \quad (\text{entspricht } \pi_2 \circ \pi_1: 1 \rightarrow 3, 2 \rightarrow 1, 3 \rightarrow 2).$$

LGS lösen mit Pivotisierung — 3-Schritt-Verfahren

Gegeben $Ax = b$, berechne P, L, R mit $PA = LR$ (Alg. 3.37). Dann:

1. Bestimme P und LR mit $PA = LR$ (Alg. 3.37).
2. Löse $Ly = Pb$ (Vorwärtssubstitution, da L untere Dreiecksmatrix).
3. Löse $Rx = y$ (Rückwärtssubstitution, da R obere Dreiecksmatrix).

Korrektheit: $Ax = P^\top(PA)x = P^\top LRx = P^\top Ly = P^\top Pb = b$ (da $P^\top P = I$).

Satz 3.40 — Existenz der LR-Zerlegung mit Pivotisierung

Zu jeder regulären Matrix $A \in \mathbb{R}^{n \times n}$ existiert eine Permutationsmatrix P_π , so dass $LR = P_\pi A$ eine LR-Zerlegung ist. Zudem kann P_π so gewählt werden, dass

$$|l_{ij}| \leq 1 \quad \forall i, j \in \{1, \dots, n\}.$$

Beweisidee: Da A regulär ist, hat die erste Spalte einen Nichtnulleintrag. Wähle k mit $|a_{k1}| = \max_i |a_{i1}| > 0$. Permutation π_1 mit $\pi_1(1) = k$ macht $(P_{\pi_1} A)_{11} \neq 0$; ein Gauss-Eliminationsschritt liefert $L_1 P_{\pi_1} A = A^{(1)}$ mit $|l_{i1}| \leq 1$. Da A regulär $\Rightarrow A^{(1)}$ regulär $\Rightarrow$ zweite Spalte von $A_{2:n,2:n}^{(1)}$ nicht null. Induktion über die Schritte; am Ende:

$$\tilde{L}_k := P_k L_k P_k^{-1}, \quad R = \tilde{L}_{n-1} \cdots \tilde{L}_1 P_0 A,$$

wobei $\tilde{L}_k$ die permutierte Version von L_k ist, deren Einträge $|\tilde{l}_{ij}| \leq 1$ erfüllen. $\square$

Python-Implementierung — LR mit Spaltenpivotsuche

```
1 import numpy as np
2
3 def lr_pivot(A):
4     n = A.shape[0]
5     A = A.astype(float).copy()
6     p = np.arange(n)          # Permutationsvektor
7     L = np.eye(n)
8
9     for k in range(n - 1):
10        # Spaltenpivotsuche: grösstes Element in Spalte k, ab Zeile k
11        i_max = k + np.argmax(np.abs(A[k:, k]))
12        # Zeilentausch im Permutationsvektor und in A
13        p[[k, i_max]] = p[[i_max, k]]
14        A[[k, i_max]] = A[[i_max, k]]
15        L[[k, i_max], :k] = L[[i_max, k], :k] # L auch tauschen
16        # Eliminationsschritt
17        for i in range(k + 1, n):
18            L[i, k] = A[i, k] / A[k, k]
19            A[i, k:] -= L[i, k] * A[k, k:]
20
21    R = A
22    return L, R, p          # PA = LR, wobei P durch p kodiert ist
23
24 def solve_lr_pivot(A, b):
25    L, R, p = lr_pivot(A)
26    Pb = b[p]              # Permutiere b
27    # Vorwaertssubstitution Ly = Pb
28    n = len(b)
29    y = np.zeros(n)
30    for i in range(n):
31        y[i] = Pb[i] - L[i, :i] @ y[:i]
32    # Rueckwaertssubstitution Rx = y
33    x = np.zeros(n)
34    for i in range(n - 1, -1, -1):
35        x[i] = (y[i] - R[i, i+1:] @ x[i+1:]) / R[i, i]
36    return x
```

Vergleich: mit vs. ohne Pivotsuche

	Ohne Pivotsuche	Mit Spaltenpivotsuche
Existenz	Nur wenn alle Hauptminoren $\neq 0$	Stets für reguläres A (Satz 3.40)
Multiplikatoren	Unbeschränkt (können sehr groß werden)	$ l_{ij} \leq 1$ garantiert
Numerische Stabilität	Kann versagen (Auslöschung)	Stabil in der Praxis
Aufwand	$O(n^3)$	$O(n^3)$ (Pivotssuche: $O(n^2)$ extra)
Ergebnis	$A = LR$ (falls existent)	$PA = LR$

Prüfungsscheckliste — Vorlesung 8

- ☐ Konditionszahl $\text{cond}(A) = \|A\| \|A^{-1}\|$ und ihre Bedeutung als Fehlerverstärkungsfaktor erklären. → *Wiederholung*
- ☐ Neumann-Reihe: Voraussetzung $\|B\| < 1$, Formel $(I - B)^{-1} = \sum B^k$ und Normschranke $\frac{1}{1-\|B\|}$ angeben. → *Wiederholung*
- ☐ Störungssatz 3.32 formulieren: Voraussetzung $\gamma < 1$, vollständige Ungleichung für $\|\Delta x\|/\|x\|$ aufschreiben. → *Satz 3.32*
- ☐ Beweis des Störungssatzes skizzieren: Faktorisierung $(A + \Delta A) = (I + B)A$, Neumann für Inverse, Fehlergleichung. → *Satz 3.32*
- ☐ Interpretation für $\gamma \ll 1$: vereinfachte Schranke und Rolle von $\text{cond}(A)$ erklären. → *Kommentare*
- ☐ Schärfe der Schranke: SPD-Beispiel mit Eigenvektoren v_1, v_n und $\Delta b = \varepsilon v_1$ durchrechnen. → *Kommentare*
- ☐ Zwei Fehlerquellen benennen: Kondition (unveränderlich) vs. Algorithmusstabilität (behebbar). → *Bsp. 3.33*
- ☐ Beispiel 3.33 erklären: warum versagt die LR-Zerlegung ohne Pivotsuche? Kleines Pivot → großer Multiplikator → Auslöschung. → *Bsp. 3.33*
- ☐ Warum löst Zeilentausch das Problem? Kleines l_{21} nach Tausch, keine Auslöschung. → *Bsp. 3.33*
- ☐ Permutationsmatrix definieren: $P_\pi = (e_{\pi(1)} | \dots | e_{\pi(n)})$; Einträge als Kronecker-Delta angeben. → *Def. 3.35*
- ☐ Lemma 3.36: vier Eigenschaften aufzählen (Einheitsvektor, Inverse= $P^\top$, Zeilenpermutation, Komposition). → *Lemma 3.36*
- ☐ Algorithmus 3.37 beschreiben: Spaltenpivotsuche, Zeilentausch, Eliminationsschritt, Ausgabe $PA = LR$. → *Alg. 3.37*
- ☐ LGS lösen mit Pivotisierung: 3 Schritte ($PA = LR$, $Ly = Pb$, $Rx = y$) und Korrektheitsbeweis via $P^\top P = I$. → *3-Schritte*
- ☐ Satz 3.40 formulieren und Beweisidee skizzieren: Existenz der Zeile mit max. Pivot → Induktion → $|l_{ij}| \leq 1$. → *Satz 3.40*

- Python: Permutationsvektor statt Matrixtausch; Zugriff `A[p[i],j]`; → *Praxis*
Vorteil (schneller).
-